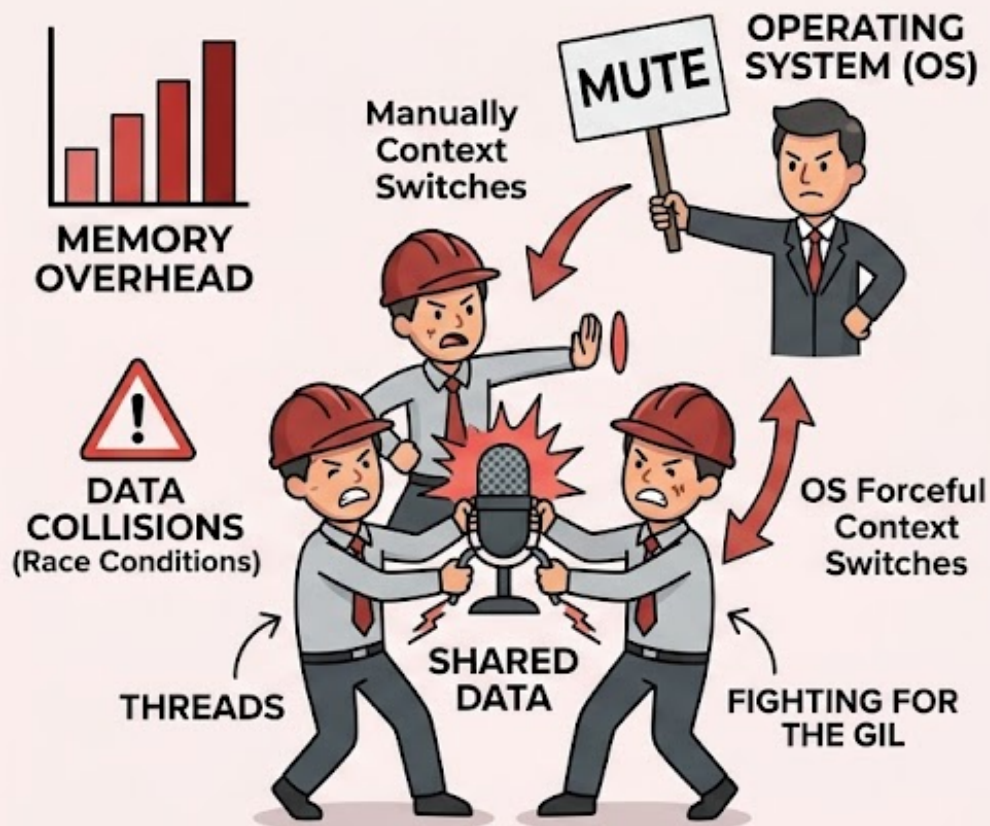


Asyncio

Coroutine

PYTHON THREADING: Preemptive & Risky



PYTHON ASYNCIO: Cooperative & Scalable



Concurrent Programming

- **Thread-based** concurrency is provided by the **threading module** and is supported by the **underlying operating system**. It is **suited to blocking I/O tasks** such reading and writing from files, sockets, and devices.
- **Process-based** concurrency is provided by the **multiprocessing module** and is also supported by the **underlying operating system**, like threads. It is suited to **CPU-bound tasks** that do not require much inter-process communication, such as compute tasks.
- **Coroutines** are an alternative that is **provided by the Python language** and runtime (standard interpreter) and further supported by the **asyncio module**. They are **suited to non-blocking I/O** with **subprocesses** and **sockets**, however, blocking I/O and CPU-bound tasks can be used in a simulated non-blocking manner using threads and processes under the covers

A Venn diagram consisting of two concentric circles. The outer circle is dark blue and contains the text 'Concurrency' and '(Threading, Async IO)'. The inner circle is light blue and contains the text 'Parallelism' and '(Multiprocessing)'. The inner circle is entirely contained within the outer circle.

Concurrency
(Threading, Async IO)

Parallelism
(Multiprocessing)

The Truth About Threads

- Threads are a features provided by an **operating system (OS)**
- Threads may indicated to OS which parts of their program **may be run in parallel.**

```
customer_threads = []
for customer in customers:
    # สร้างเธรดแยกให้ลูกค้าแต่ละคน รันฟังก์ชันกระบวนการส่วนตัว
    t = threading.Thread(target=customer_private_workflow, args=(customer,))
    customer_threads.append(t)
    t.start() # สั่งให้เธรดเริ่มทำงานขนานกันทันที
```

Benefits of Threading

- Easy of reading code - simple, **top-down** linear **sequence**
- Parallelism with **shared memory** - code **can export multiple CPUs** while still having threads share memory.
- Know-how and existing code - **best practices available**

Drawbacks of Threading

- Threading is difficult - Bugs and **race conditions**
- Threads are **resource-intensive** - require extra OS resources to create such as preallocated, per-thread stack space
- Threading can affect throughput - **high concurrency level (> 5,000 threads)**
- Threading is **inflexible** - OS share CPU time with all threads regardless of whether a thread is ready to do work or not.

What is Asynchronous Programming

- Asynchronous programming is a programming paradigm that **does not block**.
- Instead, **requests** and function calls are issued and executed **somehow** in the **background** at some **future** time
- This frees the caller to perform other activities and **handle** the results of issued calls at a **later time** when **results** are **available** or when the caller is interested

Asynchronous Programming

- The use of asynchronous techniques, such as issuing **asynchronous tasks** or **function calls**.
- Asynchronous programming is primarily used with **non-blocking I/O**, such as reading and writing from socket connections with other processes or other systems.
- **Non-blocking I/O**: Performing I/O operations via asynchronous requests and responses, **rather** than **waiting** for operations to complete.
- **Asynchronous I/O**: A shorthand that refers to **combining** asynchronous programming with non-blocking I/O.

Asynchronous Programming in Python

- Asynchronous programming in Python refers to **making requests** and **not blocking** to wait for them to complete.
- **asyncio** is short for asynchronous I/O. It is a Python library that allows us to run code using an asynchronous programming model. This lets us handle multiple I/O operations at once, while still **allowing our application to remain responsive**.

What problem is Asyncio Trying to solve?

For I/O-bound workloads, there are exactly (only!)

- Asyncio offers a safe alternative to **pre-emptive multitasking**.
 - Threads that have **bugs, race conditions**
- Asyncio offers a simple way to support **many thousands of simultaneous socket connections**, many **long-lived connections**
 - newer technologies like **WebSockets**
 - **MQTT** for Internet of Things (IoT)

Asyncio misinformation

- **X** Asyncio will make my **code blazing fast**
 - Asyncio for large numbers of concurrent **socket connection**
 - Operating system have limits threads can be created, and significant lower than the number of **socket connections**
- **X** Asyncio makes threading redundant
 - Threading able to write multi-CPU programs
- **X** Asyncio prevents all race conditions
 - Race conditions is always present with any concurrent programming
 - Asyncio control of execution is transferred between coroutines, by await

The Tower of Asyncio

Table 3-1. Features of asyncio arranged in a hierarchy; for end-user developers, the most important tiers are highlighted in bold

Level	Concept	Implementation
Tier 9	Network: streams	<code>StreamReader</code> , <code>StreamWriter</code> , <code>asyncio.open_connection()</code> , <code>asyncio.start_server()</code>
Tier 8	Network: TCP & UDP	<code>Protocol</code>
Tier 7	Network: transports	<code>BaseTransport</code>
Tier 6	Tools	<code>asyncio.Queue</code>
Tier 5	Subprocesses & threads	<code>run_in_executor()</code> , <code>asyncio.subprocess</code>
Tier 4	Tasks	<code>asyncio.Task</code> , <code>asyncio.create_task()</code>
Tier 3	Futures	<code>asyncio.Future</code>
Tier 2	Event loop	<code>asyncio.run()</code> , <code>BaseEventLoop</code>
Tier 1 (Base)	Coroutines	<code>async def</code> , <code>async with</code> , <code>async for</code> , <code>await</code>

Coroutines

Example 3-4. Async functions are functions, not coroutines

```
>>> async def f(): ❶
...     return 123
...
>>> type(f) ❷
<class 'function'>
>>> import inspect ❸
>>> inspect.iscoroutinefunction(f) ❹
True
```

Example 3-5. An async def function returns a coroutine object

```
>>> coro = f()
>>> type(coro)
<class 'coroutine'>
>>> inspect.iscoroutine(coro)
True
```

Coroutines

- A **Coroutine** is an **object** that encapsulates the ability to **resume** an underlying function that has been **suspended** before completion.

Coroutines

- A coroutine is initiated by “**sending**” it a None
- When the coroutine returns, a special kind of **exception** is raised, called **StopIteration**

Example 3-6. Coroutine internals: using send() and StopIteration

```
>>> async def f():  
...     return 123  
>>> coro = f()  
>>> try:  
...     coro.send(None) ❶  
... except StopIteration as e:  
...     print('The answer was:', e.value) ❷  
...  
The answer was: 123
```

Coroutines

- Calling **f()** produces a **coroutine**; this means we are allowed to **await** it. The value of the result variable will be **123** when **f()** completes
- `asyncio.run(main())`

Example 3-7. Using await on a coroutine

```
async def f():  
    await asyncio.sleep(1.0)  
    return 123  
  
async def main():  
    result = await f() ❶  
    return result
```

Coroutines

- 1) Our coroutine function now handles an **exception**
- 3) Here we throw() the **CancelledError** exception
- 4) As expected, we see our **cancellation message** being printed.

Example 3-9. Coroutine cancellation with CancelledError

```
>>> import asyncio
>>> async def f():
...     try:
...         while True: await asyncio.sleep(0)
...     except asyncio.CancelledError: ❶
...         print('I was cancelled!') ❷
...     else:
...         return 111
>>> coro = f()
>>> coro.send(None)
>>> coro.send(None)
>>> coro.throw(asyncio.CancelledError) ❸
I was cancelled! ❹
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration ❺
```

Coroutines

Example 3-10. For educational purposes only—don't do this!

```
>>> async def f():
...     try:
...         while True: await asyncio.sleep(0)
...     except asyncio.CancelledError:
...         print('Nope!')
...         while True: await asyncio.sleep(0) ❶
...     else:
...         return 111
>>> coro = f()
>>> coro.send(None)
>>> coro.throw(asyncio.CancelledError) ❷
Nope!
>>> coro.send(None) ❸
```

Coroutines

- 1) **Obtain** a loop.
- 2) Run the **coroutine** to **completion**

Example 3-11. Using the event loop to execute coroutines

```
>>> async def f():  
...     await asyncio.sleep(0)  
...     return 111  
>>> loop = asyncio.get_event_loop() ❶  
>>> coro = f()  
>>> loop.run_until_complete(coro) ❷  
111
```

Define, Create and Run Coroutines

```
1  # define a coroutine
2  async def custom_coro():
3      # ...
4
```

```
1  import asyncio
2  # define a coroutine
3  async def custom_coro():
4      # await another coroutine
5      await asyncio.sleep(1)
6
```

- A coroutine can then use coroutine-specific expressions within it, such as **await**, **async for**, and **async with**.

How to Create a Coroutine

```
1  import asyncio
2  # define a coroutine
3  async def custom_coro():
4      # await another coroutine
5      await asyncio.sleep(1)
6
7  # create a coroutine
8  coro = custom_coro()
```

- Once a coroutine is **defined**, it can be **created**
- This does not execute the coroutine.
- It returns a “coroutine” **object**.

How to Run a Coroutine From Python

```
1  # example of running a coroutine
2  import asyncio
3  # define a coroutine
4  async def custom_coro():
5      # await another coroutine
6      await asyncio.sleep(1)
7
8  # main coroutine
9  async def main():
10     # execute my custom coroutine
11     await custom_coro()
12
13 # start the coroutine program
14 asyncio.run(main())
15
```

Coroutines

Traditional Subroutine Pattern

Calling Method

Called Method

Code of the form:

```
#Do things
...

r =
subroutine(params)

# Do other
things
...
```

Path of execution

Call

Return

Code of the form:

```
#Do things
...

return r
```

Coroutine Pattern

Calling Method

Coroutine

Code of the form:

```
#Do things
...

c =
coroutine(params)
r = next(c)

# Do other things
...

v = next(c)

# Do other things
...

v = next(c)

# Do other things
...
```

Yield

Yield

Return

Code of the form:

```
#Do things
...

yield r

# Do other things
...

yield r

# Do other things
...

return r
```

Coroutine and Event Loop Practices

- **asyncio01.py** - Understanding `async def` and how it differs from a normal function.
- **asyncio02.py** - Seeing that calling an `async def` function creates an "Object" but does not execute it yet.
- **asyncio03.py** - Using the Event Loop to actually execute a Coroutine Object.
- **asyncio04.py** - Pausing a coroutine to let another operation finish using `await`.
- **asyncio05.py** - Showing that simply awaiting one after another is still sequential (Synchronous behavior).

Event Loop

- The **event loop** in **asyncio** handles all of the **switching** between **coroutines**, as well as catching those **StopIteration** exceptions
- You can get by **without** ever needing to work with the **event loop directly**
 - Recommended
asyncio.get_running_loop(), callable from inside the context of a coroutine
 - Discouraged
asyncio.get_event_loop(), callable from anywhere

Event Loop

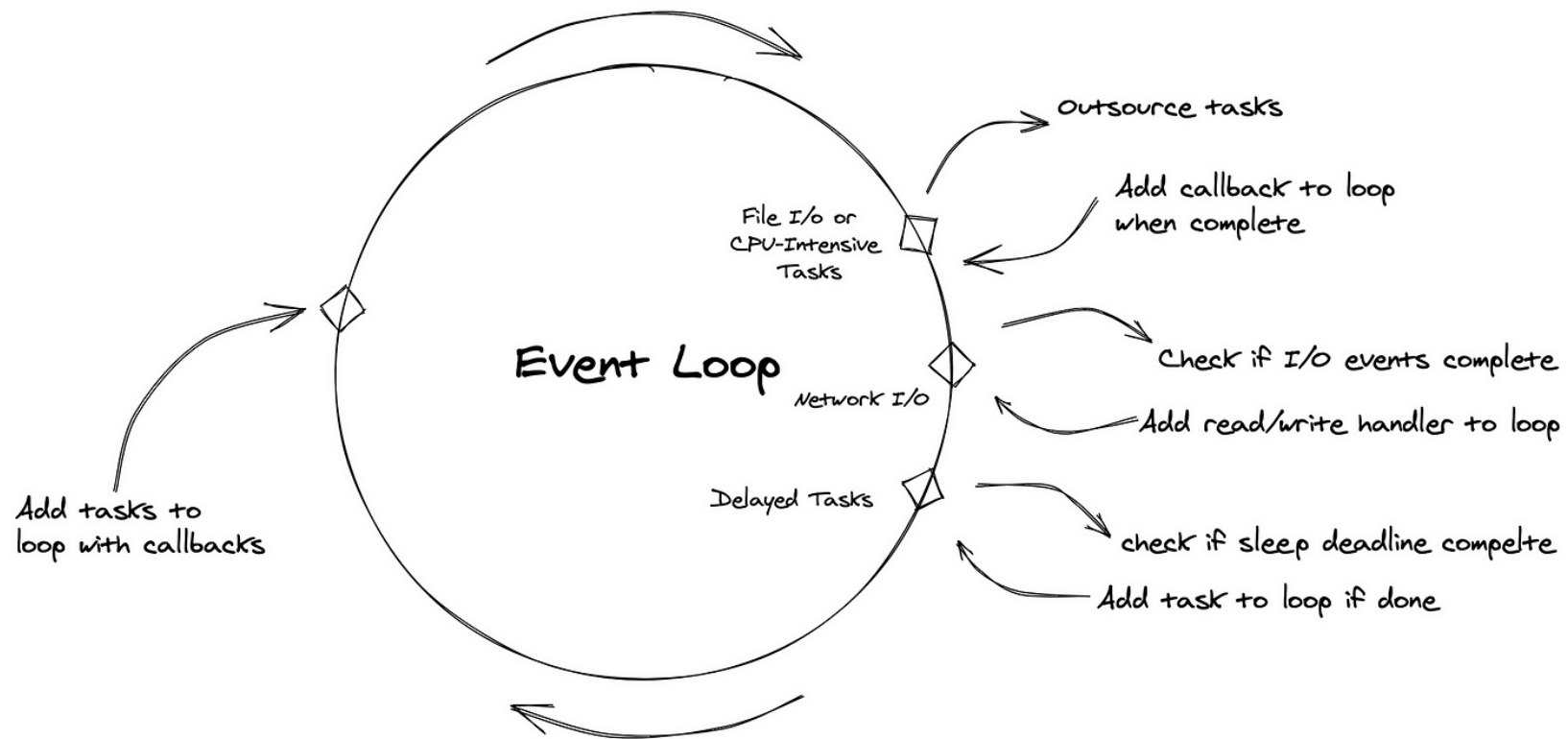
Example 3-12. Always getting the same event loop

```
>>> loop = asyncio.get_event_loop()  
>>> loop2 = asyncio.get_event_loop()  
>>> loop is loop2 ❶  
True
```

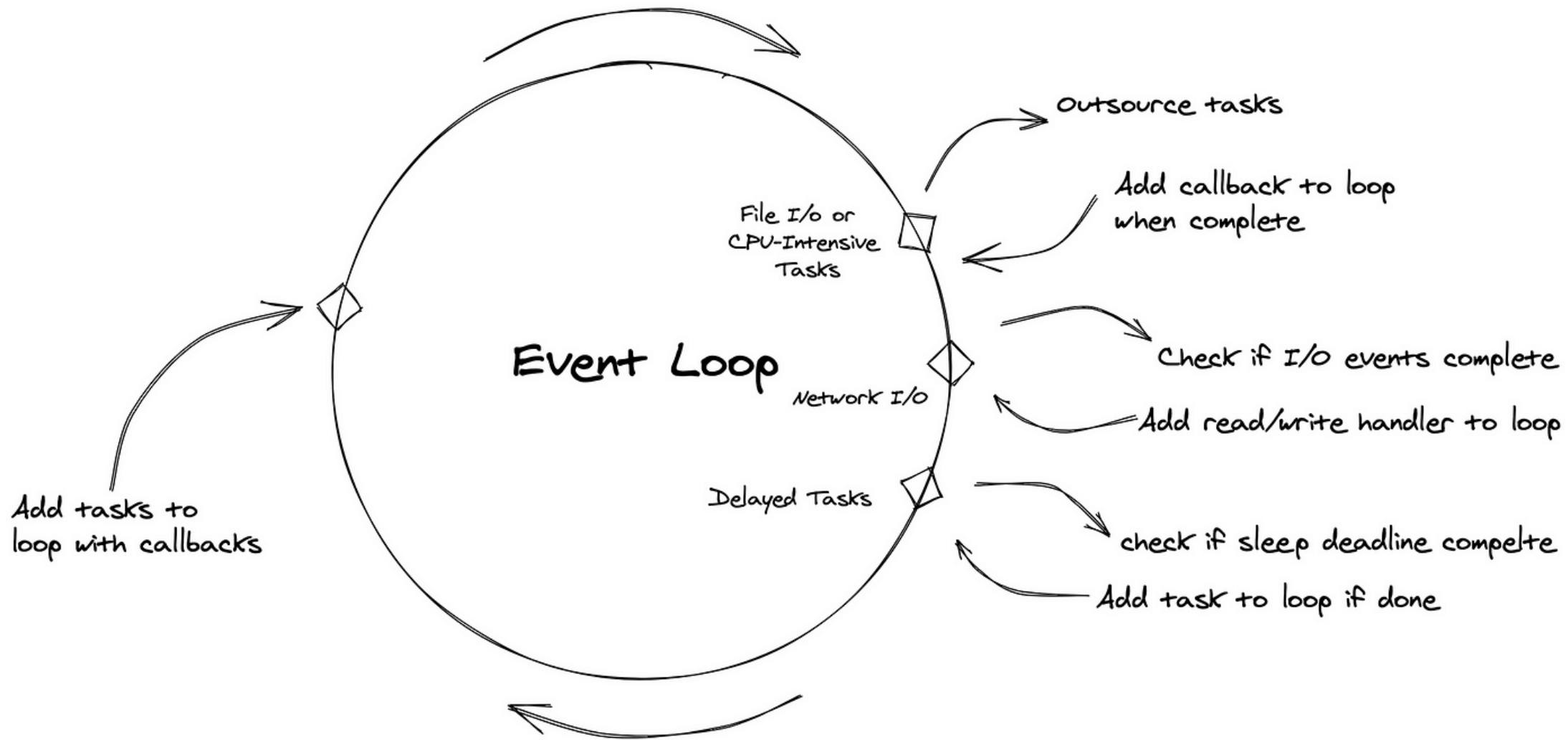
- ❶ Both identifiers, `loop` and `loop2`, refer to the same instance.

Event Loop

- The **get_event_loop()** method works only within the **same thread**
- **get_running_loop()** (the recommended method) it can be called only within the context of a coroutine, a task, or a function called of those
- **get_running_loop()** always provides the **current running event loop**



Asynchronous Task



Create and Run Asyncio Tasks

- You can **create Task** objects from **coroutines** in **asyncio programs**
- Tasks provide a handle on independently **scheduled** and **running coroutines** and allow the **task to be queried, canceled**, and **results** and exceptions to be retrieved later
- The asyncio event loop **manages tasks**. As such, all coroutines become and are managed as tasks within the event loop

What is an Asyncio Task

- A Task is an **object** that **schedules** and independently runs an asyncio coroutine
- A task is **created from a coroutine**. It requires a **coroutine object**, wraps the **coroutine**, **schedules** it for **execution**, and provides ways to interact with it
-

How to Create a Task

- Create Task With High-Level API (preferred)
- Create Task With Low-Level API

Create Task with High-Level API

- The **`asyncio.create_task()`** function takes a coroutine instance and an optional name for the task and returns an **`asyncio.Task` instance**
 - **Wrap** the coroutine in a Task instance.
 - **Schedule** the task for execution in the current event loop.
 - **Return** a Task instance

```
1 ...  
2 # create a coroutine  
3 coro = task_coroutine()  
4 # create a task from a coroutine  
5 task = asyncio.create_task(coro)
```

```
1 ...  
2 # create a task from a coroutine  
3 task = asyncio.create_task(task_coroutine())
```

When Does a Task Run?

- Although we can **schedule** a **coroutine** to run independently as a task with the **create_task()** function, it may **not run immediately**.
- The task will not execute **until the event loop has an opportunity to run**.
- This will not happen **until all other coroutines are not running** and it is the task's turn to run.

When Dose a Task Run?

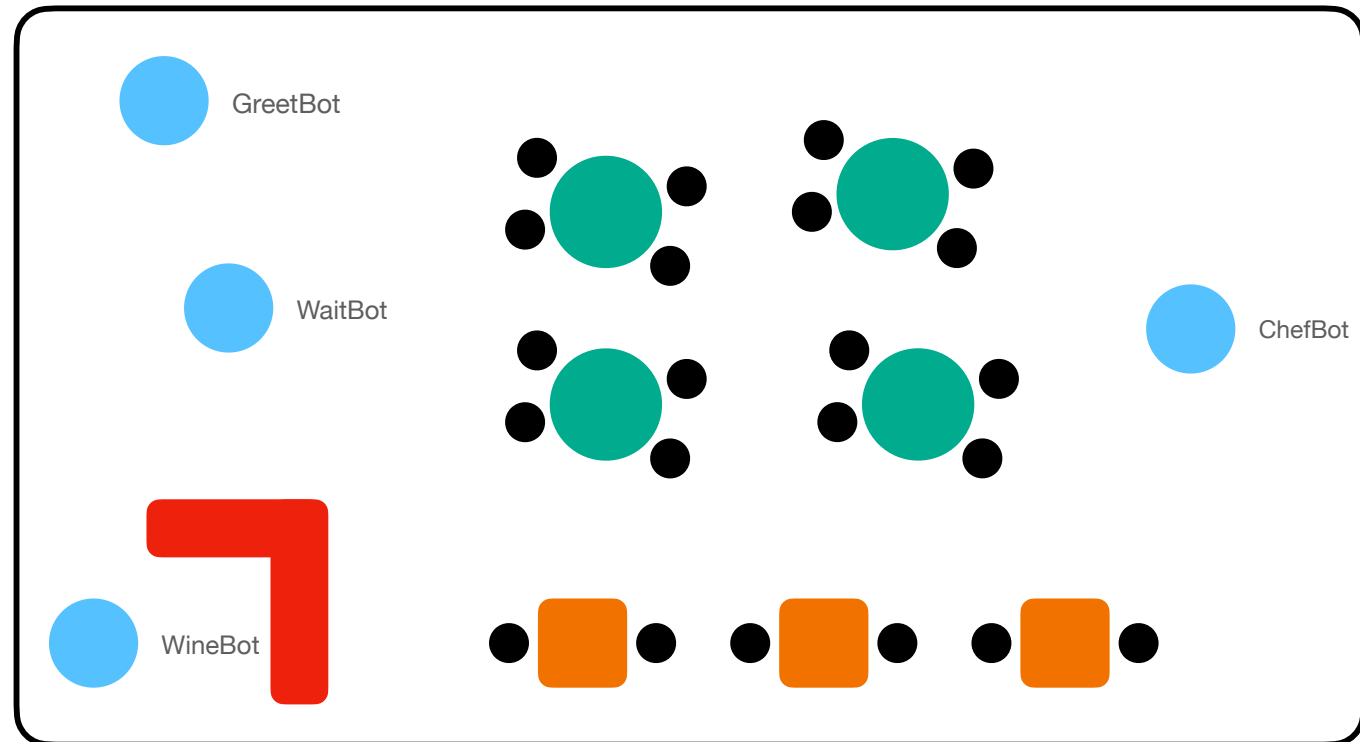
```
1 ...  
2 # create a task from a coroutine  
3 task = asyncio.create_task(task_coroutine())  
4 # await the task, allowing it to run  
5 await task
```

Tasks Practices

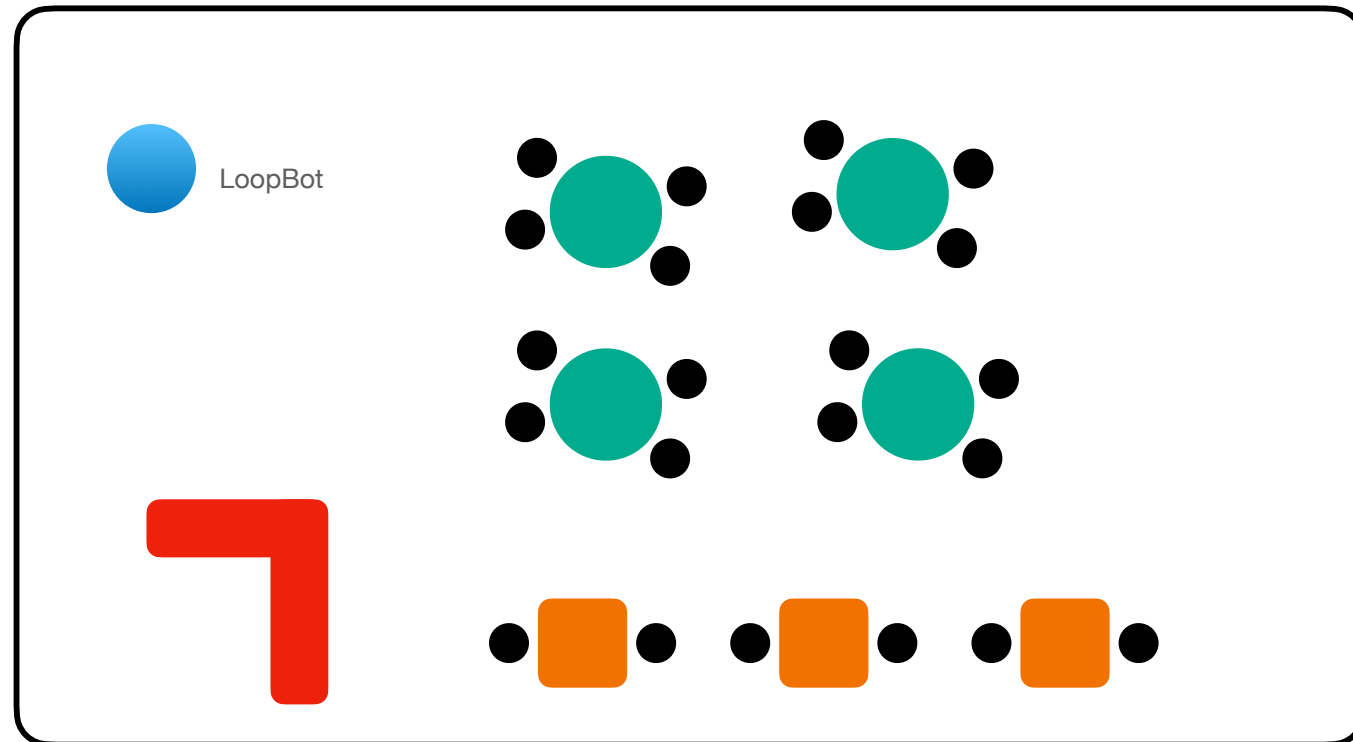
- **asyncio06.py** - Wrapping a coroutine inside `asyncio.create_task()` to schedule it to run in the background.
- **asyncio07.py** - Scheduling two distinct tasks concurrently and awaiting them individually without `gather`.
- **asyncio08.py** - Watching a single thread switch back and forth between two different workflows using `create_task`.
- **asyncio09.py** - Managing multiple generated tasks dynamically by appending them into a standard Python list.
- **asyncio10.py** - Accessing returned results from completed Task objects using `.result()` or direct assignment.

Assignment Week2

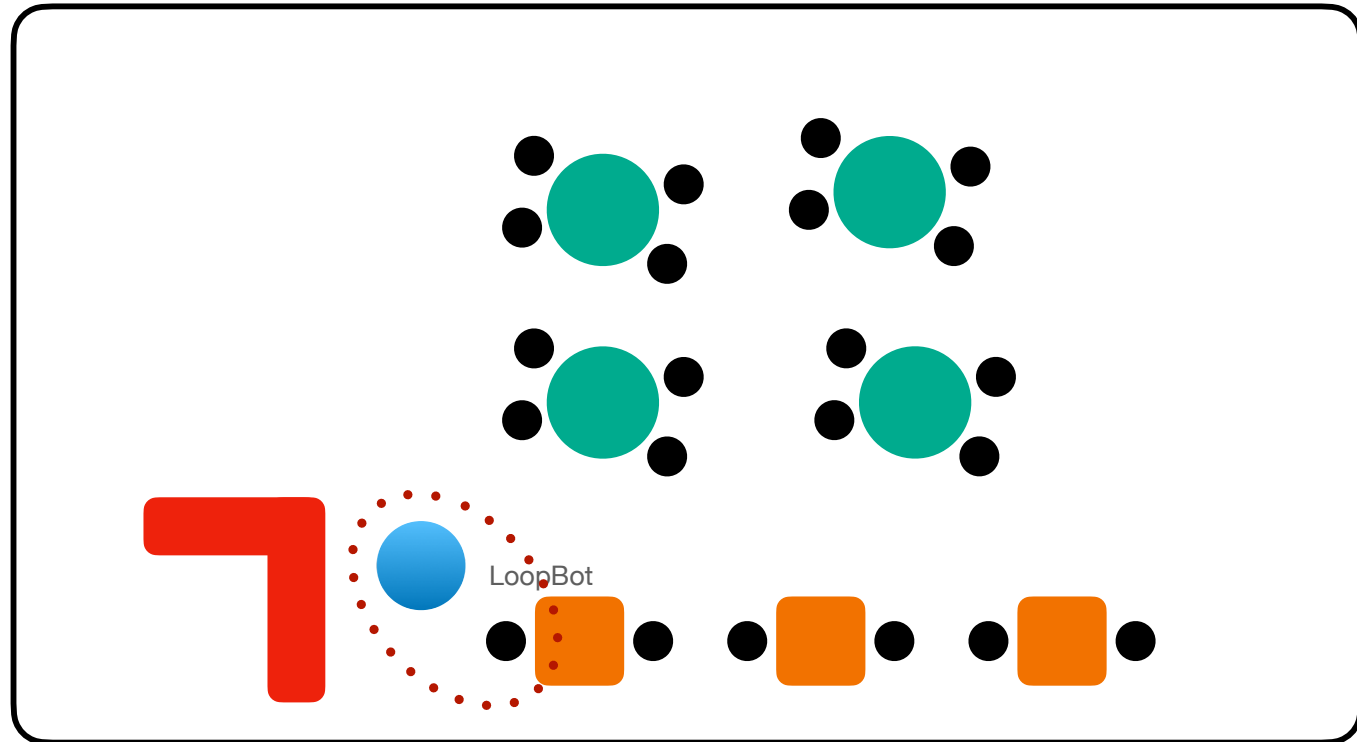
- GreetBot, **greet diners at the front desk**
- WaitBot, **wait tables and take orders**
- ChefBot, **do the cooking**
- WineBot, **manage the bar**



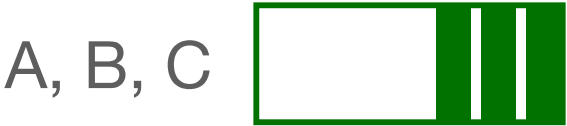
- LoopBot
 - **greet** diners at the front desk
 - **wait** tables and take orders
 - do the **cooking**
 - **manage** the bar



- LoopBot
- Effectively only if **task is short!**



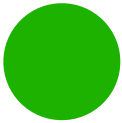
Restaurant Workflow



Greeting



Take Order



Cooking



Mini Bar

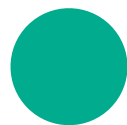
Restaurant Synchronous Workflow



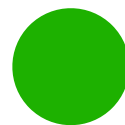
A



Greeting



Take Order



Cooking

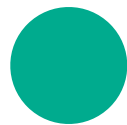


Mini Bar

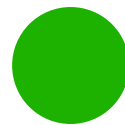
B



Greeting



Take Order



Cooking



Mini Bar

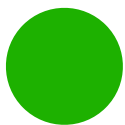
C



Greeting



Take Order

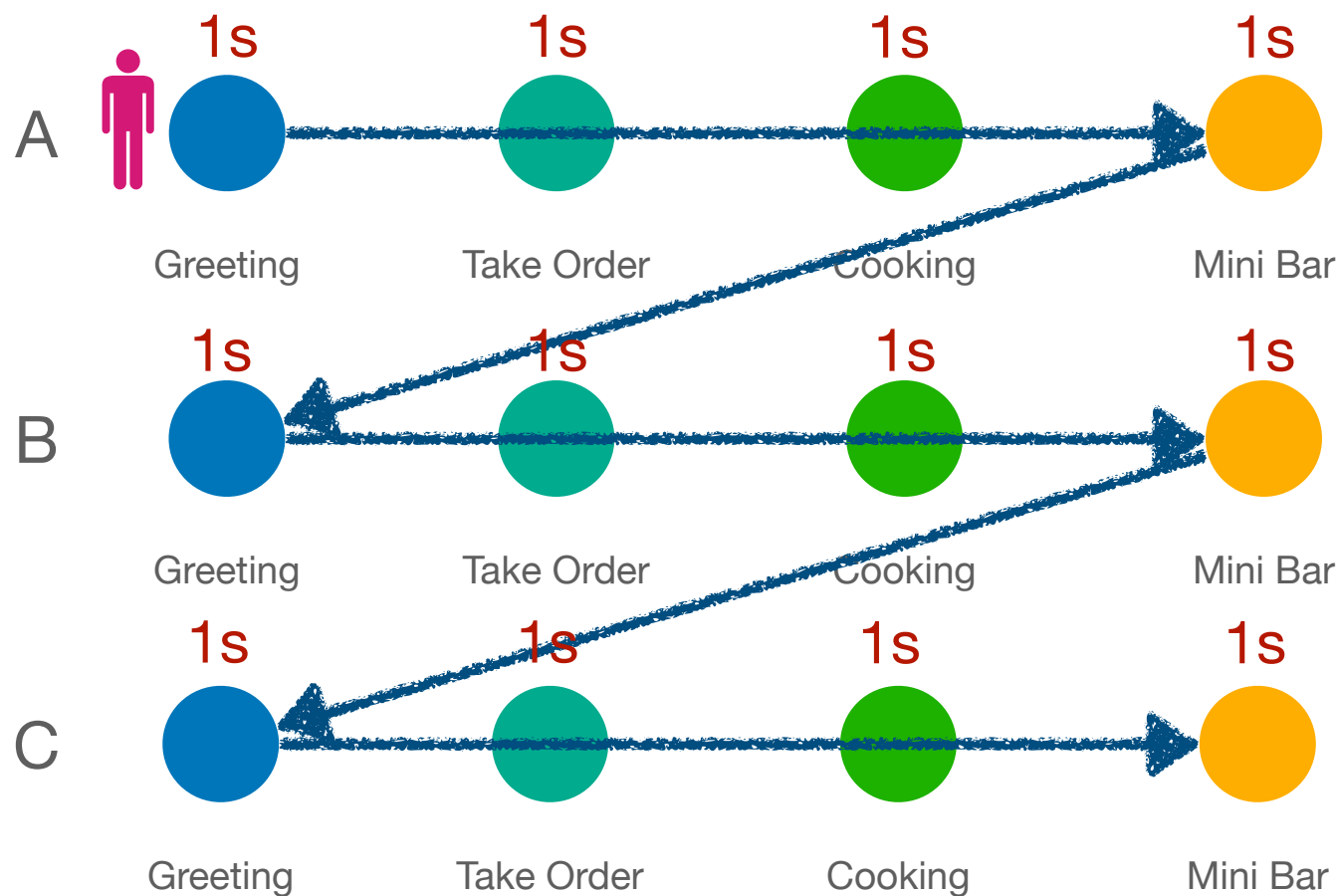


Cooking

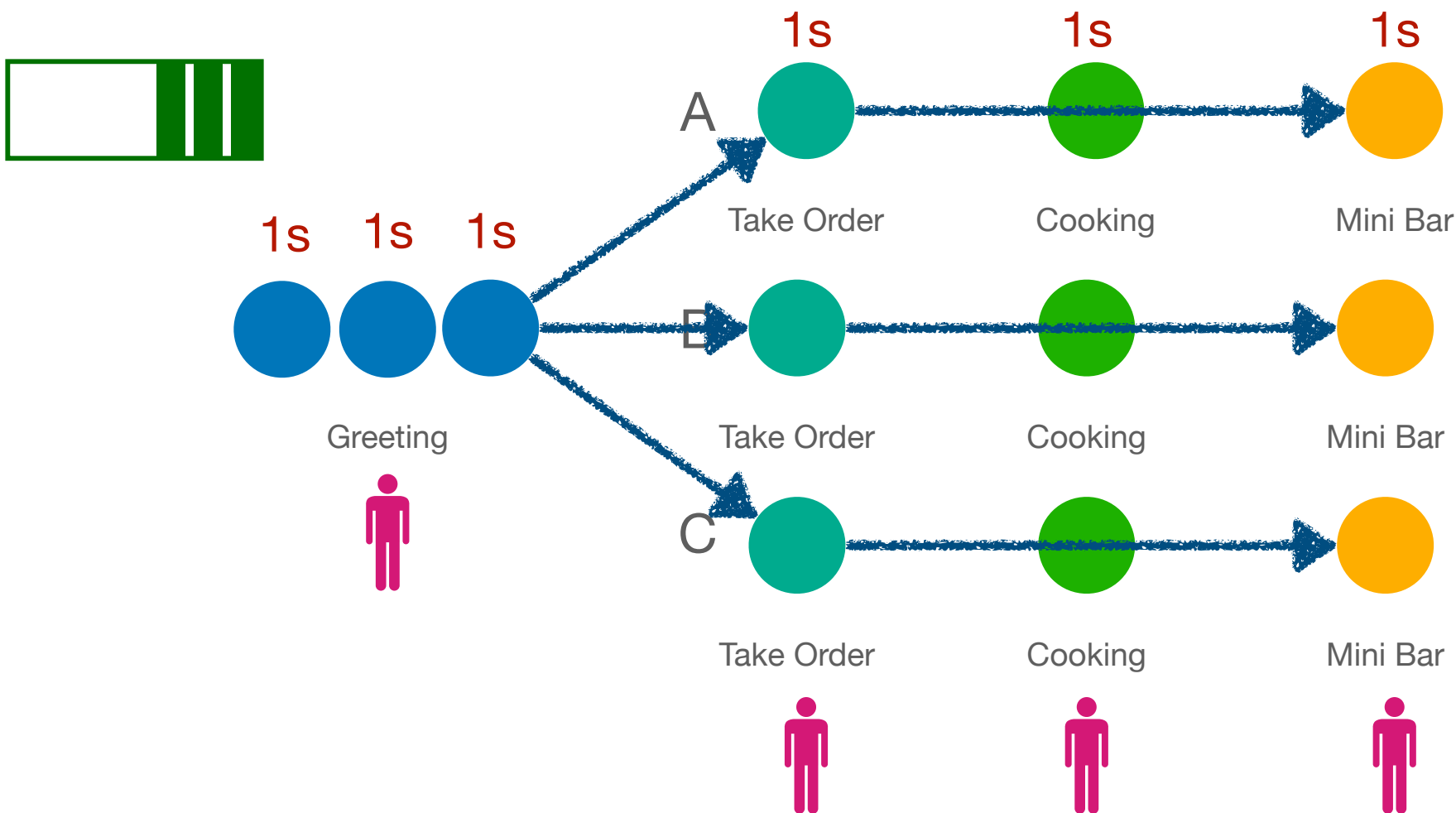


Mini Bar

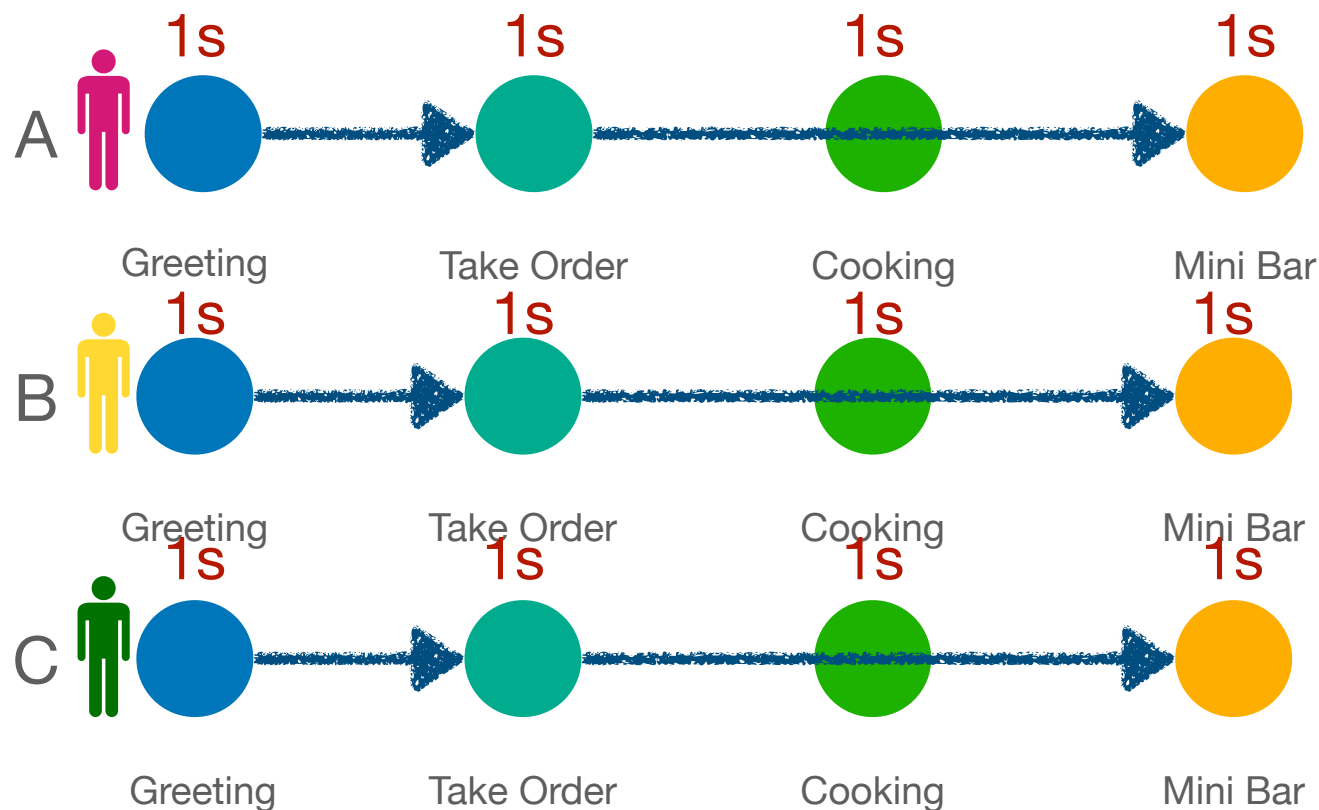
Restaurant Synchronous Workflow



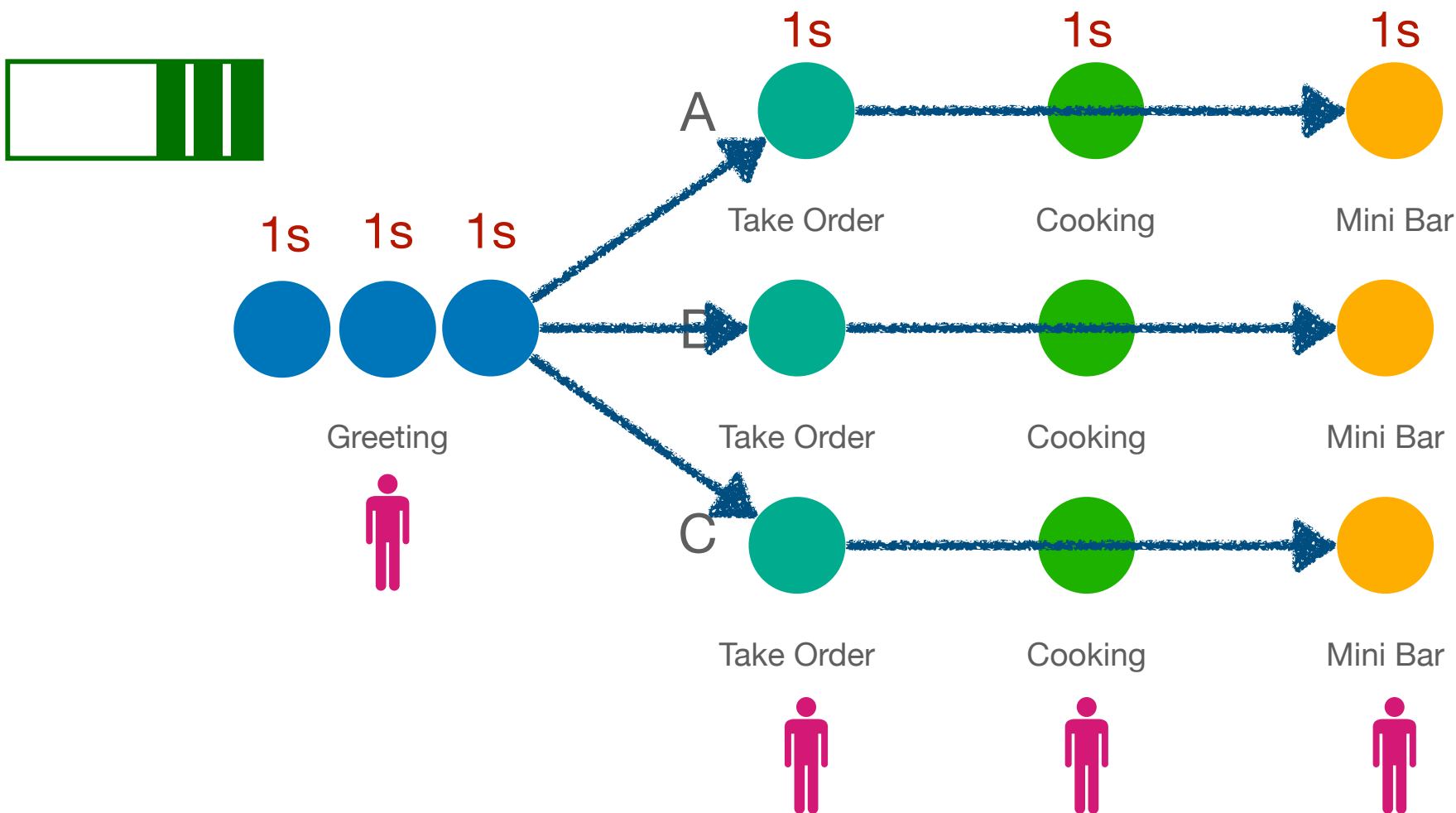
Restaurant Threading Workflow



Restaurant Multi-Process Workflow



Restaurant Multi-Process Workflow



The Restaurant Dilemma

Solving Python Concurrency through Real-World Operations



The Operational Challenge

- To manage a successful restaurant, we must execute four distinct activities simultaneously



Greet Diners

Front Desk: Welcoming guests as they arrive.



Take Orders

Wait Tables:
Communicating guest preferences.



Do Cooking

Kitchen: Heavy lifting & food preparation.

Manage Bar

Drinks: Specialized beverage creation.

Model 1: Multi-Threading

- **The "One-Radio" Illusion**
- **Concept:** Hiring 4 workers in 1 shop. They share the same space and resources.
 - If the Chef is cooking, the Bartender stands still.
 - Workers constantly fight over the radio.
 - Result: Only 1 core is used. Shifting between workers creates "overhead" without true speed.

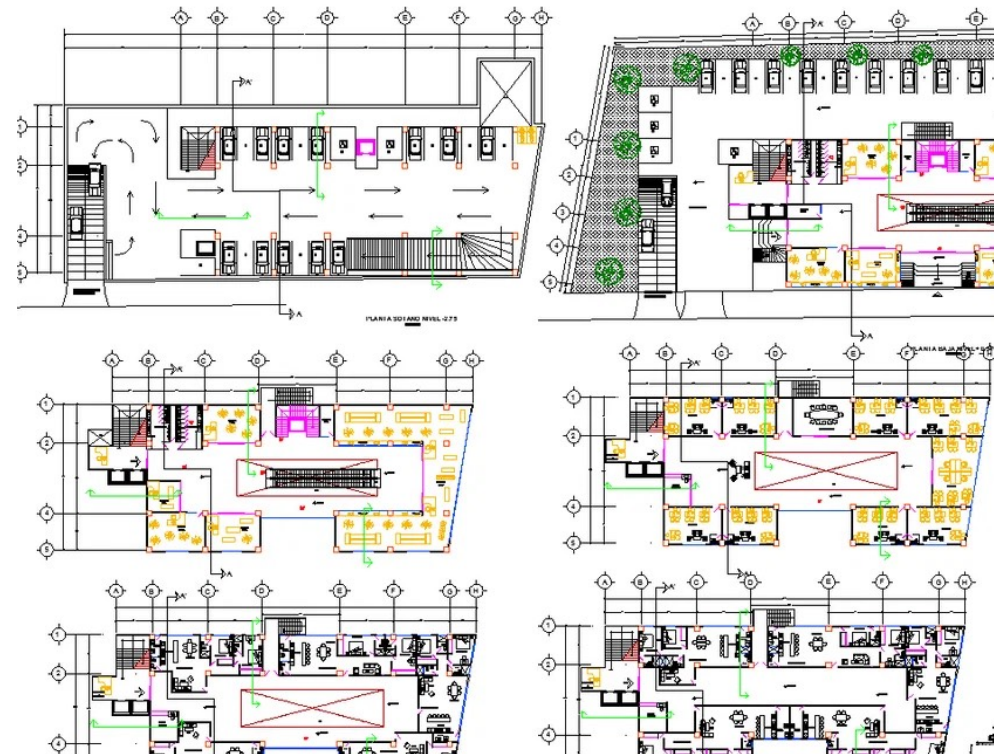
The GIL Constraint: In Python, there is only one "Communication Radio" (GIL). A worker can only move while holding it.



Model 2: Multi-Processing

- **True Expansion: Branching Out**
- **Concept:** Opening 4 separate branches (Processes). Each has its own kitchen, staff, and manager.
 - Isolation: Each branch has its own independent Radio (GIL).
 - True Parallelism: The OS puts each branch on a separate CPU core.
 - Result: Branch A cooks while Branch B takes orders at the exact same millisecond.

Cost: Higher memory (RAM) usage and complex communication between branches.



Model 3: Asyncio

- **The Smart Single-Worker**
- **Concept:** Hire 1 super-efficient worker (Single Thread). They never sit idle during "waiting time."
 - **Non-Blocking:** While the water is boiling (I/O Wait), the worker greets a new guest.
 - **Event-Driven:** When the kettle whistles (Event), they return to finish the tea.
 - **Result:** Lowest memory footprint. Perfect for handling thousands of connections.



Strategic Comparison

MODEL	WORKERS	SCALING TYPE	PRIMARY BENEFIT
Multi-Threading	Many (Shared)	I/O Bound	Simplest shared memory
Multi-Processing	Many (Isolated)	CPU Bound	Uses all CPU cores
Asyncio	Single (Event)	I/O Bound	High scalability, low RAM

restaurant_01_asyncio.py

```
Tue Jun 30 21:44:08 2026 Greeting for Customer-A ...
Tue Jun 30 21:44:09 2026 Greeting for Customer-A ...Done!
Tue Jun 30 21:44:09 2026 Greeting for Customer-B ...
Tue Jun 30 21:44:10 2026 Greeting for Customer-B ...Done!
Tue Jun 30 21:44:10 2026 Greeting for Customer-C ...
Tue Jun 30 21:44:11 2026 Greeting for Customer-C ...Done!

Tue Jun 30 21:44:11 2026 --- All customers greeted. Scheduling independent Async Tasks! ---

Tue Jun 30 21:44:11 2026 [Task-A] Taking Order ...
Tue Jun 30 21:44:11 2026 [Task-B] Taking Order ...
Tue Jun 30 21:44:11 2026 [Task-C] Taking Order ...
Tue Jun 30 21:44:12 2026 [Task-A] Taking Order ...Done!
Tue Jun 30 21:44:12 2026 [Task-A] Cooking Spaghetti ...
Tue Jun 30 21:44:12 2026 [Task-B] Taking Order ...Done!
Tue Jun 30 21:44:12 2026 [Task-B] Cooking Spaghetti ...
Tue Jun 30 21:44:12 2026 [Task-C] Taking Order ...Done!
Tue Jun 30 21:44:12 2026 [Task-C] Cooking Spaghetti ...
Tue Jun 30 21:44:13 2026 [Task-A] Cooking Spaghetti ...Done!
Tue Jun 30 21:44:13 2026 [Task-A] Manage Bar for Drink ...
Tue Jun 30 21:44:13 2026 [Task-B] Cooking Spaghetti ...Done!
Tue Jun 30 21:44:13 2026 [Task-B] Manage Bar for Drink ...
Tue Jun 30 21:44:13 2026 [Task-C] Cooking Spaghetti ...Done!
Tue Jun 30 21:44:13 2026 [Task-C] Manage Bar for Drink ...
Tue Jun 30 21:44:14 2026 [Task-A] Manage Bar for Drink ...Done!
Tue Jun 30 21:44:14 2026 [Task-A] All served!

Tue Jun 30 21:44:14 2026 [Task-B] Manage Bar for Drink ...Done!
Tue Jun 30 21:44:14 2026 [Task-B] All served!

Tue Jun 30 21:44:14 2026 [Task-C] Manage Bar for Drink ...Done!
Tue Jun 30 21:44:14 2026 [Task-C] All served!

Tue Jun 30 21:44:14 2026 Finished Entire Restaurant Operation in 6.01 seconds.
```